

Agglomerative vs Divisive Clustering, Dendrograms

What is Clustering?

Clustering is an unsupervised machine learning technique that groups a set of objects (data points, documents, images, etc.) such that objects in the same group (cluster) are more similar to each other than to those in other groups.

Hierarchical clustering is a family of clustering algorithms that build a hierarchy (tree) of clusters instead of producing a single flat partition, as algorithms like K-Means do. This hierarchy can be visualized using a diagram called a dendrogram.

There are two main strategies for building this hierarchy:

- Agglomerative (Bottom-Up) — starts with each point as its own cluster and successively merges the closest pairs.
- Divisive (Top-Down) — starts with all points in a single cluster and successively splits it into smaller clusters.

Agglomerative Clustering (Bottom-Up)

Agglomerative clustering is the more commonly used approach. It follows a greedy, bottom-up strategy:

1. Start by treating each data point (or document) as an individual cluster. If there are n data points, we begin with n clusters.
2. Compute the distance (or similarity) between every pair of clusters using a chosen distance metric (Euclidean, Manhattan, Cosine, etc.).
3. Merge the two closest clusters into a single new cluster.
4. Recompute the distances between the new cluster and all remaining clusters using a linkage criterion.
5. Repeat steps 3–4 until only one cluster remains (or a stopping criterion / desired number of clusters k is reached).
6. Record every merge and the distance at which it happened — this record is used to draw the dendrogram.

Linkage Methods (how cluster distance is computed)

Linkage	Distance Definition	Characteristic
Single Linkage	Minimum distance between any two points, one from each cluster	Can produce long, straggly 'chained' clusters
Complete Linkage	Maximum distance between any two points, one from each cluster	Produces compact, tightly bound clusters
Average Linkage	Average distance between all pairs of points across the two clusters	Balanced; less sensitive to outliers
Ward's Method	Merge that minimizes the increase in total within-cluster variance	Tends to create clusters of similar size

Divisive Clustering (Top-Down)

Divisive clustering works in the opposite direction:

1. Start with all data points in a single cluster.
2. Use a flat clustering algorithm (commonly K-Means with $k = 2$) or an optimal graph-cut method to split the cluster into two sub-clusters that are as dissimilar as possible.
3. Recursively apply the same splitting procedure to each resulting sub-cluster.
4. Stop when each cluster contains a single point, or when a desired number of clusters / stopping criterion is reached.

Divisive clustering is computationally more expensive than agglomerative clustering because, at each step, it must search for the best possible split (an NP-hard problem in the exact case), so in practice it is approximated using bisecting K-Means. For this reason, most libraries (including scikit-learn) do not provide a built-in divisive method — it is normally implemented manually, as demonstrated in the practical section below.

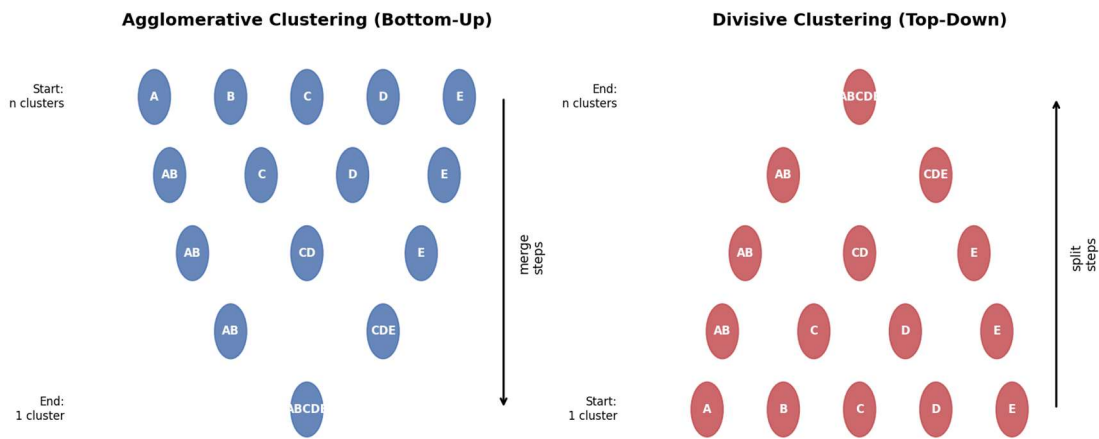


Fig 2.1 — Agglomerative (bottom-up merging) vs Divisive (top-down splitting)

Agglomerative vs Divisive: Comparison

Aspect	Agglomerative	Divisive
Approach	Bottom-up (merge)	Top-down (split)
Starting point	n singleton clusters	1 cluster with all points
Computational cost	$O(n^2 \log n)$ typical	$O(2^n)$ worst case — usually approximated
Common use	Very widely used in practice	Rarely used directly; approximated (e.g., bisecting K-Means)
Library support	scipy.cluster.hierarchy, sklearn.AgglomerativeClustering	No direct sklearn class; custom implementation needed
Accuracy of early decisions	Early merges are local and cannot be undone	Early splits are global (see all data) but also cannot be undone

Dendrograms

A dendrogram is a tree-shaped diagram that records the sequence of merges (or splits) performed by a hierarchical clustering algorithm. It is the primary visualization tool for hierarchical clustering.

- Leaves (bottom of the tree) — represent individual data points / documents.
- Internal nodes (U-shaped links) — represent a merge of two clusters; the height at which two branches join equals the distance between the merged clusters.
- Root (top of the tree) — represents the single cluster containing all points.
- Height / y-axis — a proxy for dissimilarity: the taller the link, the more dissimilar the merged clusters were.

Practical: Document Clustering

Dataset:

A corpus of 12 short documents drawn from 3 known topics (Sports, Technology, Health) is used so that clustering quality can be validated against ground truth.

#	Document (truncated)	True Topic
D01	The cricket team won the match after the batsman scored a great century.	Sports
D02	The football team played well and the coach praised the team after the match win.	Sports
D03	The tennis player won the match and secured the championship title after a tough game.	Sports
D04	The basketball team won the match after the player scored the winning point.	Sports
D05	The new smartphone has a fast processor and the camera software is very good.	Technology
D06	Artificial intelligence and machine learning software are transforming the computer industry.	Technology
D07	The laptop has a powerful processor and GPU for machine learning software tasks.	Technology
D08	Cloud computing software helps the company scale computer applications for the industry.	Technology
D09	Doctors recommend regular exercise and a balanced diet for good health of the body.	Health
D10	The hospital doctors introduced a new health treatment to reduce disease and improve health.	Health
D11	Drinking enough water and regular exercise improves body health and overall energy.	Health
D12	Doctors say meditation, exercise and proper sleep improve health and reduce stress.	Health

Procedure

1. Preprocess and vectorize the documents into numeric feature vectors using TF-IDF (Term Frequency – Inverse Document Frequency), which weighs words by their importance in a document relative to the whole corpus.
2. Compute pairwise Cosine Distance between all document vectors (cosine distance is preferred over Euclidean for text because it is insensitive to document length).
3. Apply Agglomerative Clustering using SciPy's `linkage()` function with average linkage to obtain the merge sequence.
4. Plot the Dendrogram using `scipy.cluster.hierarchy.dendrogram()`.
5. Cut the dendrogram at $k = 3$ clusters using `sklearn.AgglomerativeClustering` and evaluate with the Silhouette Score.
6. Extract top TF-IDF keywords per cluster to interpret what topic each cluster represents.
7. Implement a simple Divisive Clustering demo using recursive bisecting K-Means, for comparison.

Source Code

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import pdist, squareform

# -----
# STEP 1: Sample document corpus (3 natural topic groups)
# -----
documents = [
    # Sports
    "The cricket team won the match after the batsman scored a great century.",
    "The football team played well and the coach praised the team after the match win.",
    "The tennis player won the match and secured the championship title after a tough game.",
    "The basketball team won the match after the player scored the winning point.",
    # Technology
    "The new smartphone has a fast processor and the camera software is very good.",
    "Artificial intelligence and machine learning software are transforming the computer industry.",
    "The laptop has a powerful processor and GPU for machine learning software tasks.",
    "Cloud computing software helps the company scale computer applications for the industry.",
    # Health
    "Doctors recommend regular exercise and a balanced diet for good health of the body.",
    "The hospital doctors introduced a new health treatment to reduce disease and improve health.",
    "Drinking enough water and regular exercise improves body health and overall energy.",
    "Doctors say meditation, exercise and proper sleep improve health and reduce stress.",
]

labels_true = (["Sports"] * 4) + (["Technology"] * 4) + (["Health"] * 4)

print("=" * 70)
print("STEP 1: Corpus")
print("=" * 70)
for i, d in enumerate(documents):
    print(f"D{i+1:02d} [{labels_true[i]:10s}]: {d}")

# -----
# STEP 2: Text vectorization using TF-IDF
# -----
vectorizer = TfidfVectorizer(stop_words="english")
tfidf_matrix = vectorizer.fit_transform(documents)
tfidf_dense = tfidf_matrix.toarray()

print("\n" + "=" * 70)
print("STEP 2: TF-IDF matrix shape ->", tfidf_dense.shape)
print("=" * 70)
print("Vocabulary sample:", list(vectorizer.vocabulary_.keys())[:10])

# -----
# STEP 3: Compute pairwise distance matrix (cosine distance)
# -----
dist_matrix = pdist(tfidf_dense, metric="cosine")
square_dist = squareform(dist_matrix)
```

```

print("\n" + "=" * 70)
print("STEP 3: Pairwise cosine distance matrix (rounded)")
print("=" * 70)
np.set_printoptions(precision=2, suppress=True)
print(square_dist)

# -----
# STEP 4: Agglomerative clustering using Scipy linkage (for dendrogram)
# -----
Z = linkage(dist_matrix, method="average") # average linkage

print("\n" + "=" * 70)
print("STEP 4: Linkage matrix (scipy)")
print("=" * 70)
print("[idx1, idx2, distance, sample_count]")
print(np.round(Z, 3))

doc_labels = [f"D{i+1}\n({labels_true[i]})" for i in range(len(documents))]

plt.figure(figsize=(11, 6))
dendrogram(
    Z,
    labels=doc_labels,
    leaf_font_size=9,
    color_threshold=0.7,
)
plt.title("Dendrogram - Agglomerative Hierarchical Clustering\n(Average Linkage, Cosine Distance)")
plt.xlabel("Documents")
plt.ylabel("Cosine Distance")
plt.axhline(y=0.7, color="gray", linestyle="--", linewidth=1)
plt.tight_layout()
plt.savefig("/home/claude/practical/dendrogram.png", dpi=150)
plt.close()
print("\nDendrogram saved -> dendrogram.png")

# -----
# STEP 5: Cut the dendrogram / fit AgglomerativeClustering (k = 3)
# -----
k = 3
model = AgglomerativeClustering(n_clusters=k, metric="cosine", linkage="average")
cluster_ids = model.fit_predict(tfidf_dense)

print("\n" + "=" * 70)
print(f"STEP 5: Agglomerative Clustering result (k = {k})")
print("=" * 70)
for i, (doc, cid) in enumerate(zip(documents, cluster_ids)):
    print(f"D{i+1:02d} -> Cluster {cid} | True topic: {labels_true[i]:10s} | {doc[:55]}...")

sil = silhouette_score(tfidf_dense, cluster_ids, metric="cosine")
print(f"\nSilhouette Score (cosine): {sil:.4f}")

# -----
# STEP 6: Cluster -> Top keywords per cluster (interpretation)
# -----
print("\n" + "=" * 70)
print("STEP 6: Top TF-IDF keywords per cluster")
print("=" * 70)
terms = np.array(vectorizer.get_feature_names_out())
for c in range(k):
    idxs = np.where(cluster_ids == c)[0]

```

```

cluster_tfidf = tfidf_dense[idxs].mean(axis=0)
top_idx = cluster_tfidf.argsort()[::-1][:6]
print(f"Cluster {c}: {'', '.join(terms[top_idx])}")

# -----
# STEP 7: Divisive clustering (conceptual demo) via repeated bisection
#         scikit-learn has no built-in divisive clustering, so we
#         demonstrate the idea using recursive KMeans(k=2) bisection.
# -----
from sklearn.cluster import KMeans

def divisive_bisect(X, doc_idxs, depth, max_depth, results):
    if depth == max_depth or len(doc_idxs) <= 1:
        results.append(doc_idxs)
        return
    km = KMeans(n_clusters=2, n_init=10, random_state=42)
    sub = X[doc_idxs]
    lbl = km.fit_predict(sub)
    left = [doc_idxs[i] for i in range(len(doc_idxs)) if lbl[i] == 0]
    right = [doc_idxs[i] for i in range(len(doc_idxs)) if lbl[i] == 1]
    divisive_bisect(X, left, depth + 1, max_depth, results)
    divisive_bisect(X, right, depth + 1, max_depth, results)

results = []
all_idxs = list(range(len(documents)))
divisive_bisect(tfidf_dense, all_idxs, depth=0, max_depth=2, results=results)

print("\n" + "=" * 70)
print("STEP 7: Divisive Clustering demo (recursive bisecting K-Means)")
print("=" * 70)
for cnum, group in enumerate(results):
    docs_in_group = [f"D{i+1}({labels_true[i]})" for i in group]
    print(f"Divisive Cluster {cnum}: {docs_in_group}")

print("\nDone.")

```

3.6 Output

Step 1 & 2 — Corpus loaded and converted to a TF-IDF matrix of shape (12 documents × 71 vocabulary terms).

```

TF-IDF matrix shape -> (12, 71)
Vocabulary sample: ['cricket', 'team', 'won', 'match', 'batsman',
                    'scored', 'great', 'century', 'football', 'played']

```

Step 3 — Pairwise cosine distance matrix (excerpt). Notice distances are low (~0.6-0.8) within a topic group and close to 1.0 (maximally dissimilar) across topic groups:

	D01	D02	D03	D04	D05	D06	...
D01 [	0.00	0.77	0.85	0.62	1.00	1.00	...
D02 [	0.77	0.00	0.94	0.77	1.00	1.00	...
D03 [	0.85	0.94	0.00	0.74	1.00	1.00	...
D04 [	0.62	0.77	0.74	0.00	1.00	1.00	...
D09 [	1.00	1.00	1.00	1.00	0.88	1.00	... 0.00 0.80 0.65 0.78
D10 [	1.00	1.00	1.00	1.00	0.89	1.00	... 0.80 0.00 0.88 0.64

Step 4 — Dendrogram produced from the linkage matrix:

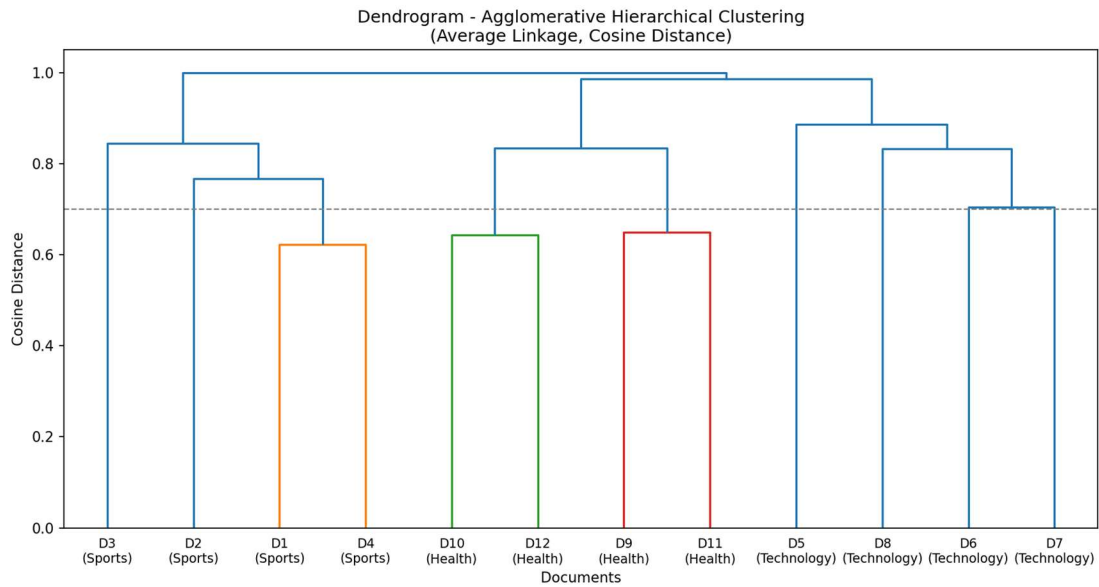


Fig 3.1 — Dendrogram of the 12-document corpus (Average Linkage, Cosine Distance). Three clean topic clusters emerge: Sports, Health, Technology.

Step 5 — Cutting the dendrogram at $k = 3$ (via `AgglomerativeClustering`) reproduces the true topic groups exactly:

Document	True Topic	Predicted Cluster
D01–D04	Sports	Cluster 1
D05–D08	Technology	Cluster 0
D09–D12	Health	Cluster 2

Silhouette Score (cosine) = 0.1947 — a positive score confirming that documents are, on average, more similar to members of their own cluster than to members of other clusters.

Step 6 — Top TF-IDF keywords per cluster (used to label / interpret each cluster):

Cluster	Top Keywords	Interpreted Topic
Cluster 0	software, processor, learning, machine, industry, computer	Technology
Cluster 1	team, match, won, scored, player, winning	Sports
Cluster 2	health, exercise, doctors, regular, body, reduce	Health

Step 7 — Divisive Clustering demo (recursive bisecting K-Means, depth = 2). The first split already separates Health from the rest; the second level of splits separates Sports and Technology:

```

Divisive Cluster 0: [D09(Health), D10(Health), D11(Health), D12(Health)]
Divisive Cluster 1: [D05(Technology)]
Divisive Cluster 2: [D01(Sports), D02(Sports), D03(Sports), D04(Sports)]
Divisive Cluster 3: [D06(Technology), D07(Technology), D08(Technology)]

```